MSCIKDF: An Atomic, Seed-Storage-Free Identity Primitive for Autonomous Digital Entities

Jian Sheng Wang
jason@mscikdf.com
Montreal, Canada

December 2025

Abstract

We introduce the *Atomic Digital Identity* (ADI), a new deterministic and seed-storage-free identity primitive designed to satisfy the structural requirements of emerging Digital Entities such as AI agents, robots, IoT devices, and autonomous services. An ADI provides a unified identity root that deterministically derives curve- and context-isolated key streams across heterogeneous cryptographic ecosystems without ever storing a master seed or long-term secret.

We present **MSCIKDF** as a concrete construction of ADI. The primitive supports *stateless secret rotation*, *enterprise-controlled revocation*, and *unifies deterministic asset control with native end-to-end encrypted (E2EE) communication capabilities*, alongside seamless *post-quantum migration*, while maintaining strict domain separation and cryptographic isolation across all derived keys. Identity recovery is performed solely from a mnemonic–passphrase pair; the root entropy value never appears at rest and is never persisted.

We provide the full specification, security rationale, and a complete reference implementation in Rust, and we demonstrate applications in multi-chain wallets, enterprise KMS, autonomous device identity, secure messaging, and AI identity frameworks.

1 Introduction

1.1 The Emergence of Digital Entities

We introduce the notion of a *Digital Entity* (or *Digital Being*): any autonomous computational actor—including AI agents, robots, IoT devices, micro-services, digital twins, or decentralized services—that requires a persistent and verifiable cryptographic identity

capable of operating across heterogeneous environments, devices, and protocols.

A Digital Entity must possess the following eight capabilities:

1. **Deterministic root identity** that persists across devices, lifecycles, instantiations, and execution environments.
2. **Curve- and domain-isolated key streams** enabling secure signing, encryption, authentication, and protocol-specific operations without cross-domain leakage.
3. **Native end-to-end encrypted (E2EE) communication capability**, ensuring that independent encryption keys may be derived for every peer, session, or context while maintaining deterministic identity roots.
4. **Seed-storage-free recoverability**, guaranteeing that no master secret or root seed is ever stored at rest while remaining deterministically reconstructible from user-held material.
5. **Stateless secret rotation**, allowing unlimited passphrase updates without rekeying, re-deriving, or altering the underlying identity.
6. **Enterprise-controllable revocation**, enabling organizations to unilaterally revoke an identity without server-side state, key management infrastructure, or cooperation from the Digital Entity.
7. **Ability to hold and transfer digital assets**, enabling autonomous agents to maintain custody, execute transactions, and interact with value-bearing protocols in a cryptographically verifiable manner.
8. **Post-quantum migration capability**, permitting augmentation with PQC material without reconstructing root identity, migrating wallets, or rotating legacy secrets.

No existing cryptographic system simultaneously satisfies these eight requirements, motivating the construction of a new identity primitive. To reason about these requirements more precisely, we introduce two additional concepts that formalize the identity model underlying Digital Entities.

Crypto Entities. A *Crypto Entity* is the cryptographic instantiation of a Digital Entity: a deterministic identity object capable of deriving isolated signing, encryption, authentication, E2EE, and asset-control material across arbitrary curves and domains, all without persisting any long-term secret. A Crypto Entity must be fully reconstructible from user-held material, support stateless rotation and enterprise revocation, and remain extensible to future post-quantum augmentation. In this sense, Crypto Entities represent the minimal cryptographically meaningful abstraction of Digital Entities.

Atomic Digital Identity (ADI). We define the underlying identity primitive enabling Crypto Entities as an *Atomic Digital Identity (ADI)*: a deterministic, seed-storage-free identity root from which all curve-specific and domain-specific key streams are derived via labeled and isolated HKDF expansion. An ADI serves as the stable identity substrate of a

Crypto Entity, and a single ADI must support signing, encryption, E2EE communication, authentication, asset interaction, and PQC augmentation without rekeying, re-seeding, or state migration.

MSCIKDF as a construction of ADI. The MSCIKDF primitive introduced in this work provides a concrete realization of ADI that satisfies the structural requirements above. It offers deterministic identity, multi-curve support, stateless rotation, enterprise-controlled revocation, seed-storage-free recoverability, and PQC-ready extension, making it suitable as the unified identity layer for Digital Entities across wallets, secure messaging systems, enterprise KMS deployments, AI agents, robotic platforms, and IoT ecosystems.

1.2 Background and Motivation

Modern cryptographic ecosystems face increasing fragmentation across chains, curves, protocols, and identity models. Wallets, secure messaging systems, enterprise KMS deployments, and emerging AI frameworks each maintain separate identity infrastructures, resulting in:

- **Multi-curve and multi-chain identity fragmentation**, with no unified root supporting diverse ecosystems.
- **Security risks from seed persistence**, including disk leakage, latent backups, memory scraping, and BDO-style attacks.
- **Lack of stateless revocation mechanisms**, particularly in enterprise settings where KMS relies on server-side state.
- **Absence of deterministic identity for AI agents/robots/IoT devices**, preventing cryptographic continuity and asset ownership.
- **No unified identity root across wallets, KMS, and AI systems**, forcing disjoint seed hierarchies.
- **High-cost PQC transition**, as traditional identity systems require rekeying or state migration.

These challenges motivate the design of a unified, seed-storage-free identity primitive capable of supporting both cryptographic and operational requirements of Digital Entities and their Crypto Entities.

1.3 Limitations of Existing Systems

Existing identity mechanisms partially address aspects of the problem but fail to meet the full set of requirements needed for Atomic Digital Identity:

- **BIP39** provides mnemonic-based seed generation but supports only a single curve and offers no rotation or revocation mechanism. Once the seed is leaked or stored, all derived keys are permanently at risk.
- **SLIP-10** extends derivation to multiple curves but does not eliminate seed storage and lacks domain-isolated semantics across heterogeneous protocols.

- **HKDF** offers strong extract–expand properties but lacks mnemonic semantics, identity models, and an integrated multi-curve / multi-context framework.
- **SR25519** supports domain separation within its own ecosystem but cannot generalize across curves, asset protocols, or independent E2EE / KMS deployments.
- **Enterprise KMS solutions** depend on server-side state and long-lived master keys, making seed-storage-free operation and stateless revocation infeasible.

None of these systems provide seed-storage-free identity, stateless rotation, enterprise revocation, or deterministic identity for Digital Entities and their Crypto Entities. This gap motivates the construction of a new primitive realizing ADI.

1.4 Our Contributions

This work introduces **MSCIKDF**, a new *cryptographic identity primitive* designed to realize Atomic Digital Identity (ADI) and to satisfy the structural requirements of Digital Entities in modern multi-curve cryptographic ecosystems. Our contributions include:

- **Formalization of Crypto Entities and ADI.** We introduce Crypto Entities as the cryptographic instantiation of Digital Entities, and Atomic Digital Identity (ADI) as the seed-storage-free identity primitive underpinning them.
- **Multi-curve deterministic identity root**, enabling consistent identity across arbitrary curves and protocols from a single ADI.
- **Seed-storage-free identity**, ensuring that no master secret is ever persisted while remaining recoverable from a mnemonic–passphrase pair.
- **Stateless secret rotation**, allowing unlimited passphrase updates without altering derived keys or requiring stateful key migration.
- **Native E2EE support** via deterministic X25519 key derivation with explicit domain isolation, suitable for messaging and agent-to-agent communication.
- **Enterprise-grade revocation** via a composable passphrase-binding mechanism that enables unilateral revocation without server-side key storage.
- **Domain- and context-isolated key derivation** using structured HKDF labeling, ensuring cryptographic independence across curves, applications, and protocols.
- **Deterministic identity for Digital Entities and Crypto Entities**, including AI agents, robots, IoT devices, and long-lived autonomous services.
- **Native asset-holding capability** for Digital Entities interacting with on-chain and off-chain protocols.
- **PQC-ready migration**, enabling post-quantum augmentation without rekeying, changing mnemonics, or losing identity continuity.
- **Reference implementation in Rust**, including test vectors and FFI bindings for multi-platform deployment.

- **Threat model and security analysis**, covering confidentiality, domain isolation, rotation soundness, and enterprise revocation.

2 Preliminaries

This section introduces notation, the generation of the 128-bit *Root Entropy Value* (REV), the domain-separated HKDF expansion mechanism, the asymmetric primitives used by MSCIKDF, the sealing construction based on AES-GCM-SIV, and the threat model under which the Atomic Digital Identity (ADI) primitive is analyzed.

2.1 Notation

We use the following notation throughout the paper:

- $\{0,1\}^n$ denotes the set of all binary strings of length n.
- REV denotes the 128-bit *Root Entropy Value*, sampled uniformly at random.
- $\mathsf{HKDF_Expand}(\mathsf{key}, \mathsf{info}, \ell)$ denotes HKDF expansion using SHA-256 as the underlying hash function.
- $\mathsf{AES\text{-}GCM\text{-}SIV_Enc}_K(N, M)$ and $\mathsf{AES\text{-}GCM\text{-}SIV_Dec}_K(N, C)$ denote authenticated encryption and decryption with key K and nonce N.
- $\|$ denotes byte-string concatenation.
- $\mathsf{Ed25519}$, $\mathsf{Secp256k1}$, and $\mathsf{X25519}$ refer to their respective key-generation and signing/encryption primitives.
- DS denotes a domain-separation label used as HKDF `info`.
- $\mathsf{pass_user}$ and $\mathsf{pass_ent}$ denote the user and enterprise passphrase components, respectively.
- $\mathsf{pass_real} = \mathsf{pass_user} \| 0x00 \| \mathsf{pass_ent}$.
- xID denotes the deterministic X25519 encryption identity derived in MSCIKDF.

2.2 Root Entropy Value (REV)

MSCIKDF begins from a uniformly sampled 128-bit string, denoted

$$\mathsf{REV} \leftarrow \{0,1\}^{128}.$$

The REV serves as the high-entropy root of the *Atomic Digital Identity* (ADI) primitive: all signing, encryption, authentication, and asset-control key material is derived from this value.
Unlike traditional hierarchical deterministic systems, REV is:

- never stored in persistent form,
- never exposed to the user or to applications,

- recoverable only through the sealing mechanism described in Section 2.5.

Its generation requires no structural assumptions beyond uniform randomness and is compatible with future post-quantum augmentation.

Rationale for a 128-bit REV. We set $\mathsf{REV} \in \{0,1\}^{128}$ because 2^{128} meets or exceeds the security level of all underlying primitives used in MSCIKDF (AES-GCM-SIV, Ed25519, X25519, Secp256k1). This entropy size allows the sealed artifact to be efficiently encoded into standard mnemonic formats (e.g., typically 24 words under BIP39) or other compact representations without excessive storage overhead.

2.3 HKDF with Domain Separation

MSCIKDF uses HKDF-SHA256 in *expand-only* mode to derive curve-specific and context-specific seeds:

$$s_{\mathsf{curve}} = \mathsf{HKDF_Expand}(\mathsf{REV}, \mathsf{DS}_{\mathsf{curve}}, 32),$$

where $\mathsf{DS}_{\mathsf{curve}}$ is a unique domain label identifying the curve or protocol.
Representative labels include:

$$\mathsf{DS}_{\mathsf{Ed25519}}, \quad \mathsf{DS}_{\mathsf{Secp256k1}}, \quad \mathsf{DS}_{\mathsf{X25519}}, \quad \mathsf{DS}_{\mathsf{PQC}}.$$

Domain separation ensures that:

- seeds for different curves never collide,
- derived keys remain independent under arbitrary composition,
- corruption in one domain does not compromise another.

Within the ADI model, HKDF functions as a domain-separated PRF that deterministically expands the root entropy into isolated identity material for each curve and context.

2.4 Asymmetric Primitives

MSCIKDF relies on the following asymmetric constructions, each derived via domain-separated HKDF expansion from the same REV.

Ed25519. Used for deterministic signatures and identity authentication. The derived seed is used directly as a 32-byte Ed25519 seed.

Secp256k1. Used for ECDSA signatures across EVM-based, Bitcoin-based, and Cosmos-based protocols. The derived 32-byte seed is reduced modulo the curve order.

X25519. Used for Diffie–Hellman key exchange and E2EE communication. The derived seed is clamped according to RFC 7748.

xIdentity (xID). MSCIKDF derives a deterministic X25519-based encryption identity, called *xIdentity* (xID), via a dedicated domain label $\mathsf{DS}_{\mathsf{xID}}$. The xID serves as the long-term E2EE identity of a Crypto Entity and supports deterministic but unlinkable per-session key derivation via HKDF.

2.5 Sealing via AES-GCM-SIV

To enable seed-storage-free operation without managing random nonces, MSCIKDF employs **AES-GCM-SIV** (defined in RFC 8452) as its Deterministic Authenticated Encryption (DAE) scheme.
Unlike standard AES-GCM, which fails catastrophically if a nonce is reused, AES-GCM-SIV provides *Nonce-Misuse Resistance* (NMR). It derives the internal initialization vector from both the nonce and the plaintext itself (SIV stands for Synthetic Initialization Vector). The sealing operation is defined as:

$$C = \mathsf{AES\text{-}GCM\text{-}SIV_Enc}_K(0, \mathsf{REV}),$$

where:

- K is the 256-bit key derived from the passphrase via Argon2id.
- N_{fixed} is a system-wide fixed nonce (e.g., all zeros), serving as a domain constant.
- REV is the 128-bit identity root.

Because AES-GCM-SIV is a deterministic function of (K, N, M), reusing the fixed nonce is cryptographically safe as long as the key-message pair is unique. This construction eliminates the need to store a random nonce, allowing the 32-byte output (16-byte ciphertext + 16-byte authentication tag) to fit perfectly within standard encoding limits.

2.6 Threat Model

We assume a polynomial-time adversary $\mathcal{A}$ with the following capabilities:

Adversarial capabilities.

- **Full access to the mnemonic.** $\mathcal{A}$ may obtain the BIP39-encoded ciphertext via leakage or compromise.
- **Full visibility of all public keys and signatures.** MSCIKDF maintains security under arbitrary public-key exposure.
- **No knowledge of the passphrase.** Without $K = \text{Argon2id}(\mathsf{pass_real})$, the adversary cannot recover REV.
- **Multi-device compromise.** Compromise of any set of devices does not reveal REV, since no device stores any seed or key material.
- **Domain cross-contamination attempts.** $\mathcal{A}$ cannot derive one curve's seed from another domain due to HKDF domain isolation.
- **Enterprise adversary model.** Possession of $\mathsf{pass_ent}$ alone does not allow impersonation; the user adversary with only $\mathsf{pass_user}$ and the mnemonic cannot bypass revocation after $\mathsf{pass_ent}$ is deleted.

System assumptions.

- No seeds, private keys, or REV are ever stored.
- All sensitive values exist only ephemerally and are erased immediately after use.
- AES-GCM-SIV provides *Nonce-Misuse Resistance* (NMR) and standard authenticated encryption security (IND-CCA2) even when nonces are repeated.
- Argon2id resists offline brute-force attacks on the passphrase layer.

This threat model aligns with traditional assumptions used in hierarchical deterministic wallets, secure messaging protocols (e.g., Signal), and enterprise KMS deployments, but extends them to support seed-storage-free operation, deterministic identity, and stateless enterprise revocation.

Notes on Encoding and KDF Layering:

BIP39 Encoding Semantics. MSCIKDF uses the BIP39 wordlist solely as an *encoding* format for the sealed ADI root. The mnemonic does *not* represent a BIP39 seed and is not compatible with hierarchical deterministic wallets.
KDF Layering. MSCIKDF uses two independent KDF layers:

- Argon2id protects human passphrases against offline guessing;
- HKDF-SHA256 acts as a domain-separated PRF for deterministic multi-curve and multi-context identity derivation once REV is recovered.

Memory hardness is applied only at the passphrase layer, since REV already provides 128 bits of uniform entropy.

3 MSCIKDF Overview

This section provides a high-level description of the MSCIKDF architecture, its relationship to the *Atomic Digital Identity* (ADI) primitive, and the trust boundaries under which the system operates. MSCIKDF is a fully seed-storage-free construction: the 128-bit Root Entropy Value (REV) is never persisted, never exported, and exists only ephemerally during derivation of cryptographic identity material.

3.1 System Architecture

MSCIKDF consists of two coupled but independently secure pipelines: (1) the deterministic identity pipeline, which instantiates an ADI, and (2) the sealing pipeline, which enables mnemonic-based recoverability without exposing or storing any secret material.

Deterministic Identity Pipeline (ADI Construction).

$$\begin{aligned}\mathsf{REV} &\xrightarrow{\mathsf{HKDF+DS}} \text{Curve- and Context-Specific Seeds}\\ &\xrightarrow{\text{KeyGen}} \text{Private/Public Keypairs}\\ &\xrightarrow{\text{AddressEnc / IdentityEnc}} \text{Identity Material.}\end{aligned}$$

More explicitly:

$$\mathsf{REV} \longrightarrow \{s_{\mathsf{Ed25519}},\, s_{\mathsf{Secp256k1}},\, s_{\mathsf{X25519}},\, s_{\mathsf{xID}},\, \ldots\}$$

$$\longrightarrow \{\text{Ed25519 keys, Secp256k1 keys, X25519 DH keys, xIdentity (xID), } \ldots\}.$$

Each component is derived via domain-separated HKDF from the same 128-bit root, forming a complete *Atomic Digital Identity (ADI)*: a deterministic, multi-curve, cryptographically isolated identity root suitable for Digital Entities such as AI agents, robots, and autonomous services.
The ADI is the cryptographic representation of a *Crypto Entity*.

xIdentity (XID). MSCIKDF derives a deterministic X25519-based encryption identity (xID) that serves as the long-term E2EE identity for a Crypto Entity. Session keys are derived via HKDF using unique context labels, providing unlinkable per-session encryption while retaining deterministic root identity.

Mnemonic Sealing Pipeline.

$$\mathsf{REV} + \mathsf{pass_real} \xrightarrow{\mathsf{AES\text{-}GCM_Enc}} \mathsf{sealed} \xrightarrow{\text{BIP39Encode}} \mathsf{mnemonic}.$$

Recovery proceeds as:

$$\mathsf{mnemonic} + \mathsf{pass_real} \xrightarrow{\mathsf{AES\text{-}GCM_Dec}} \mathsf{REV}.$$

Thus the reversible pipeline is:

$$\mathsf{REV} \longleftrightarrow \mathsf{sealed} \longleftrightarrow \mathsf{mnemonic}.$$

The mnemonic is *not* a seed, nor a BIP39-compatible secret; it is a UX-friendly encoding of sealed REV. No entropy is derived from the mnemonic itself.

Seed-Storage-Free Property. MSCIKDF never persists:

- the REV,
- any derived seeds,
- any private keys,
- any intermediate secret state.

Identity material is reconstructed only at runtime from a mnemonic – passphrase pair and is immediately erased after use.

BIP39 Compatibility Disclaimer. The mnemonic uses the BIP39 wordlist solely as an encoding mechanism for the sealed ADI root. It is not a BIP39 seed, does not interact with HD wallet derivation, and is not compatible with any SLIP-0010 or BIP32 construction. The encoding is representational only.

3.2 Design Goals

MSCIKDF is designed to satisfy the eight structural requirements of Digital Entities introduced in Section 1. The system achieves the following goals:

1. **Deterministic multi-curve identity:** Ed25519, Secp256k1, X25519, and future curve parameters are derived deterministically from a single ADI root, ensuring identity persistence across lifecycles.
2. **Strict domain isolation:** All seeds are expanded via HKDF-SHA256 with structured domain labels, providing cryptographic independence so that key exposure in one context never compromises others.
3. **Native E2EE identity capability:** The system derives a deterministic X25519-based xIdentity to serve as a static root for Diffie–Hellman negotiations, enabling secure, forward-secret communication channels without seed persistence.
4. **Seed-storage-free operation:** No long-term secret (seed or private key) is ever persisted. Identity is reconstructed only ephemerally from the sealed artifact.
5. **Stateless secret rotation:** Unlimited passphrase updates are supported without rekeying, re-deriving, or altering the underlying identity root.
6. **Stateless enterprise revocation:** Identity is revoked solely by deleting the enterprise passphrase component, requiring no server-side state or key management infrastructure.
7. **Asset-holding capability:** Digital Entities can deterministically sign transactions, hold addresses, and interact with value-bearing protocols across heterogeneous chains.
8. **PQC migratability:** PQC key material can be deterministically derived from the same ADI without updating mnemonics or altering identity continuity.

These goals position MSCIKDF as a general-purpose identity foundation beyond traditional wallets or KMS, suitable for AI agents, robotics, decentralized infrastructures, and autonomous systems.

3.3 Trust Boundaries

MSCIKDF distinguishes clearly among three actors:

User. Holds:

- the mnemonic (sealed ciphertext), and
- the user passphrase component pass_user.

The user never receives or stores REV or any seed.

Enterprise (Optional). Supplies:

- an enterprise-controlled passphrase component $\mathsf{pass_ent}$.

The enterprise can:

- revoke identity by deleting $\mathsf{pass_ent}$,
- enforce lifecycle control without storing keys,
- prevent impersonation (cannot reconstruct identity alone).

Adversary. May:

- obtain the mnemonic,
- observe all public keys,
- intercept signatures or E2EE messages.

Cannot:

- derive REV without both passphrase components,
- violate domain-isolated derivation between curves,
- bypass enterprise revocation,
- reconstruct xID or other private keys without sealed REV.

The system assumes no persistent secrets, no trusted hardware, and relies entirely on KDF isolation, AES-GCM-SIV IND-CCA security, and the secrecy of passphrase components.

4 Formal Specification of MSCIKDF

This section provides the complete formal specification of MSCIKDF as a construction of an *Atomic Digital Identity* (ADI). All operations rely exclusively on standardized primitives (HKDF-SHA256, AES-GCM-SIV, Argon2id, Ed25519, Secp256k1, X25519). The system is strictly seed-storage-free: the Root Entropy Value (REV) exists only ephemerally during computation.

4.1 Atomic Digital Identity (ADI)

We define an *Atomic Digital Identity (ADI)* as a triple:

$$\mathsf{ADI} = (\mathsf{REV}, F_{\mathsf{DS}}, \mathsf{Seal})$$

where:

- $\mathsf{REV} \in \{0,1\}^{128}$ is uniformly sampled and never stored,
- F_{DS} is a domain-separated HKDF expansion function,

- Seal is the authenticated encryption scheme binding the ADI to a mnemonic–passphrase pair.

Given domain labels $\mathsf{DS}_1, \ldots, \mathsf{DS}_N$, an ADI deterministically generates all curve-, context-, and protocol-specific seeds:

$$s_i = F_{\mathsf{DS}}(\mathsf{REV}, \mathsf{DS}_i).$$

An ADI therefore forms the cryptographic basis of a *Crypto Entity*, which aggregates all deterministic keys, addresses, and encryption identities derived from a single root. MSCIKDF provides a concrete construction of ADI.

4.2 Generation of the Root Entropy Value

The ADI root is generated from a 128-bit uniformly random value.

Algorithm 1 GenerateREV

```
REV ← {0,1}^128
return REV
```

No additional structure is imposed; REV is the sole entropy source for all subsequent deterministic derivations.

4.3 Passphrase-Sealing Scheme

MSCIKDF ensures that REV is never stored by sealing it under a passphrase-derived AES-GCM-SIV key. A memory-hard KDF (Argon2id) derives this key from the user/enterprise passphrase.

4.3.1 Sealing Key Derivation

Let:

$$\mathsf{pass} = \mathsf{pass}_{\mathsf{user}} \parallel 0x00 \parallel \mathsf{pass}_{\mathsf{ent}}$$

denote the composable passphrase. The sealing key is derived as:

$$K = \mathsf{Argon2id}(\text{password} = \mathsf{pass},\ \text{salt} = \texttt{"MSCIKDF-seal-key"},\ t,\ m,\ p).$$

Argon2id is used only for human-chosen inputs; it never interacts with HKDF.

4.3.2 Sealing Procedure

We utilize a deterministic authenticated encryption primitive, denoted $\mathsf{DAE_Enc}$, which incorporates a fixed system-wide parameter or synthetic IV logic to obviate the storage of a random nonce.

Algorithm 2 Seal(REV, pass)

```
K ← Argon2id(pass, "MSCIKDF-seal-key")
C ← DAE_Enc_K(REV)                          ▷ Deterministically encrypt REV
sealed ← C                                  ▷ Payload contains ciphertext and tag only
mnemonic ← BIP39Encode(sealed)
return mnemonic
```

4.3.3 Unsealing Procedure

Algorithm 3 Unseal(mnemonic, pass)

```
sealed ← BIP39Decode(mnemonic)
C ← sealed
K ← Argon2id(pass, "MSCIKDF-seal-key")
REV ← DAE_Dec_K(C)
if authentication fails then
    return ⊥
end if
return REV
```

4.4 Domain-Separated Multi-Curve Seed Derivation

Each cryptographic domain derives a pseudorandom seed from REV:

$$s_{\mathsf{curve}} = \mathsf{HKDF_Expand}(\mathsf{REV}, \mathsf{DS}_{\mathsf{curve}}, 32).$$

Representative domain labels:

$$\begin{aligned} \mathsf{DS}_{\mathsf{ed25519}} &= \texttt{MSCIKDF-ed25519-seed-v1}, \\ \mathsf{DS}_{\mathsf{secp256k1}} &= \texttt{MSCIKDF-secp256k1-seed-v1}, \\ \mathsf{DS}_{\mathsf{x25519}} &= \texttt{MSCIKDF-x25519-seed-v1} \end{aligned}$$

xIdentity (xID). The derived X25519 key pair serves as the deterministic *xIdentity* (xID) of the Crypto Entity. Unlike ephemeral session keys, the xID is a long-term, stable public identifier used for establishing E2EE channels (e.g., as a static Diffie–Hellman root). It allows the Crypto Entity to be discoverable and addressable across contexts.

4.5 Seed Normalization Rules

- **Ed25519:** HKDF output is used directly as 32-byte seed.
- **Secp256k1:**
$$k = s_{\mathsf{secp256k1}} \bmod n.$$
- **X25519 / xID:** Clamping per RFC 7748:
$$k[0] \,\&= 248, \quad k[31] \,\&= 127, \quad k[31] \,|= 64.$$

4.6 Deterministic Key Generation

Ed25519 KeyGen.

$$(sk, pk) = \mathsf{Ed25519_KeyGen}(k_{\mathsf{ed}})$$

Secp256k1 KeyGen.

$$pk = k_{\mathsf{k1}} \cdot G$$

X25519 / xID KeyGen.

$$pk = \mathsf{X25519}(k_{\mathsf{x}}, \mathsf{base})$$

5 Seed-Storage-Free Identity

A defining property of MSCIKDF is that the system never stores, persists, or exposes any form of seed material. The Root Entropy Value (REV), all curve-specific seeds, and all private keys exist only ephemerally during derivation. This distinguishes MSCIKDF from hierarchical deterministic (HD) wallets and traditional key-management systems, which rely on persistent seed material as a long-term secret.

5.1 Definition

Definition 1 (Seed-Storage-Free Identity)**.** *A cryptographic identity construction is said to be* seed-storage-free *if the following conditions hold:*

1. *The system never stores the root entropy value* REV *or any derived curve-specific seed in persistent storage.*
2. *The system never exposes* REV *or seed material to the user, any external interface, or any application-layer subsystem.*
3. *All seed material is reconstructed solely from a sealed entropy blob using a passphrase-derived key.*
4. *After key derivation, all secret values (including* REV*, seeds, and private keys) are immediately erased from memory.*

An identity satisfying these properties guarantees that no device compromise, backup leak, or filesystem forensics can reveal seed material, because such material never exists at rest.

In MSCIKDF, all determinism originates from REV, but REV is only obtainable through authenticated decryption of the sealed entropy blob. Thus, the mnemonic is not a seed; it is merely an encoding of AES-GCM-SIV ciphertext.

5.2 Theorem 1 (No Seed Persistence)

Theorem 5.1 (No Seed Persistence)**.** *Let* sealed *be the AES-GCM-SIV encryption of the REV under a passphrase-derived key K. Assuming AES-GCM-SIV is IND-CPA secure and provides ciphertext integrity, the sealed value leaks no information about* REV*, and therefore no seed material is recoverable without knowledge of K.*

Proof Sketch. AES-GCM-SIV satisfies IND-CPA confidentiality and INT-CTXT authenticity. Thus, for any ciphertext

$$\mathsf{sealed} = \mathsf{AES\text{-}GCM\text{-}SIV_Enc}_K(0, \mathsf{REV}),$$

the distribution of sealed is computationally indistinguishable from the encryption of any other 128-bit value under the same key.
Therefore:

- sealed behaves as a pseudorandom value to any adversary lacking K.
- No information about REV is leaked, since recovering REV requires successful AES-GCM-SIV authentication.
- Curve-specific seeds $s_{\mathsf{curve}} = \mathsf{HKDF_Expand}(\mathsf{REV}, \mathsf{DS}, 32)$ are pseudorandom under the pseudorandomness of HKDF and the secrecy of REV.

Thus, MSCIKDF persists *no seeds*: neither the REV nor any curve-specific seed is ever stored or derivable from the mnemonic alone. □

5.3 Comparison to Seed-Based Wallets

Most hierarchical deterministic wallet systems—BIP39, SLIP-10, and other seed-centric constructions—rely on storing a long-term master seed. This approach introduces several systemic risks:

1. **Disk persistence:** Seeds inevitably appear on disk through application files, backups, logs, or virtual memory snapshots.
2. **Seed leakage:** If the seed leaks, all derived private keys across all curves and chains are permanently compromised.
3. **Backup exposure:** Traditional backups preserve seeds indefinitely, creating long-lived attack surfaces and forensic recoverability.
4. **Key rotation requires seed replacement:** Changing passphrase or credential parameters requires constructing an entirely new seed and migrating all associated assets.

MSCIKDF eliminates these structural weaknesses:

- **No stored seed exists to be recovered.** Even full-disk forensics cannot reveal REV or any derived seed because these values never appear on disk.
- **Mnemonic leakage alone is harmless.** The mnemonic contains only AES-GCM-SIV ciphertext, not seed material. Without the passphrase-derived key, recovery of REV is infeasible.
- **Seed rotation is unnecessary.** Secret rotation is *stateless*: the underlying identity remains fixed, while security can be strengthened by changing only the passphrase component.
- **Cross-domain compromise is prevented.** HD wallets fail catastrophically if their master seed is exposed. In MSCIKDF, no master seed exists.

In effect, MSCIKDF replaces the traditional notion of a persistently stored seed with a reversible, encrypted representation of the root entropy value, dramatically reducing the attack surface of deterministic identity systems.

6 Passphrase Binding

A defining property of MSCIKDF is the strict separation between *identity* and *authorization*. Identity is determined solely by the Atomic Digital Identity (ADI) root REV, while authorization is determined by the passphrase that encrypts and protects REV. This separation enables two capabilities uncommon in classical seed-based systems:

- **stateless secret rotation** — arbitrary updates of the passphrase without altering identity, and
- **stateless revocation** — unilateral enterprise-driven invalidation of a Crypto Entity without storing any key material.

The core invariant is:

$$\text{Identity} = f(\mathsf{REV}) \qquad \text{and} \qquad \text{Authorization} = \mathsf{pass}.$$

Thus identity is immutable (as long as REV is fixed), while authorization is freely mutable.

6.1 Passphrase Binding

MSCIKDF composes the passphrase from user- and enterprise-controlled fragments:

$$\mathsf{pass} = \mathsf{pass}_{\mathsf{user}} \parallel 0x00 \parallel \mathsf{pass}_{\mathsf{ent}}.$$

The delimiter $0x00$ ensures injectivity even under variable-length components.
The sealing key is derived via:

$$K = \mathsf{Argon2id}(\mathsf{pass},\ \texttt{"MSCIKDF-seal-key"},\ t, m, p).$$

If the enterprise component $\mathsf{pass}_{\mathsf{ent}}$ is removed, the user can no longer reconstruct the full passphrase and therefore cannot decrypt the sealed REV. This provides unilateral revocation without storing any secrets.

6.2 Stateless Secret Rotation

Because the mnemonic encodes only:

$$\mathsf{mnemonic} = \mathsf{BIP39Encode}(N \| C),$$

where

$$C = \mathsf{AES\text{-}GCM\text{-}SIV_Enc}_{K(\mathsf{pass})}(0, \mathsf{REV}),$$

any passphrase update simply re-seals the same REV under a new passphrase:

$$\mathsf{mnemonic}_{\text{new}} = \mathsf{Seal}(\mathsf{REV}, \mathsf{pass}_{\text{new}}).$$

Identity-derived material remains unchanged because it depends only on the domain-separated HKDF expansions of REV.

6.3 Algorithm 8: RotateSecret

This procedure involves no persistent state, no interaction with a server, and no key regeneration. The old mnemonic may be safely discarded.

Algorithm 4 RotateSecret($\mathsf{mnemonic}_{\text{old}}, \mathsf{pass}_{\text{old}}, \mathsf{pass}_{\text{new}}$)

```
REV ← Unseal(mnemonic_old, pass_old)        ▷ Recover REV ephemerally
mnemonic_new ← Seal(REV, pass_new)          ▷ Re-seal under new passphrase
return mnemonic_new
```

6.4 Theorem 2: Rotation Soundness

Theorem 6.1 (Rotation Soundness)**.** *Let* REV *be the ADI root of a Crypto Entity. For any two passphrases* pass_1 *and* pass_2*,*

$$\mathsf{Unseal}(\mathsf{Seal}(\mathsf{REV}, \mathsf{pass}_1), \mathsf{pass}_1) = \mathsf{Unseal}(\mathsf{Seal}(\mathsf{REV}, \mathsf{pass}_2), \mathsf{pass}_2) = \mathsf{REV}.$$

Thus all derived seeds, private keys, public keys, addresses, xID encryption keys, and PQC expansions remain invariant under secret rotation.

Proof. For any passphrase pass, sealing produces:

$$C = \mathsf{AES\text{-}GCM\text{-}SIV_Enc}_{K(\mathsf{pass})}(0, \mathsf{REV}),$$

with $K(\mathsf{pass})$ obtained via a deterministic Argon2id computation. Correctness of AES-GCM-SIV guarantees:

$$\mathsf{AES\text{-}GCM\text{-}SIV_Dec}_{K(\mathsf{pass})}(0, C) = \mathsf{REV}.$$

Since domain-separated HKDF produces:

$$s_i = \mathsf{HKDF_Expand}(\mathsf{REV}, \mathsf{DS}_i, 32),$$

all seeds are identical under any passphrase. Thus, all keys and addresses based on s_i remain unchanged. □

7 Enterprise-Grade Revocation

Enterprise environments require the ability to revoke a Digital Entity' s authorization without interacting with the user, without storing any long-term secret, and without relying on trusted hardware. MSCIKDF achieves these properties through its composable passphrase-binding construction, which cleanly separates:

$$\underbrace{\text{identity}}_{\text{determined by } \mathsf{REV}} \quad \text{from} \quad \underbrace{\text{authorization}}_{\text{determined by passphrase}}.$$

Identity is immutable as long as the ADI root REV remains the same; authorization is mutable and revocable without state.

7.1 Passphrase Binding Model

Let the user contribute a passphrase fragment $\mathsf{pass}_{\text{user}}$ and the enterprise contribute an independent fragment $\mathsf{pass}_{\text{ent}}$. These two components are bound into a single effective authorization secret:

$$\mathsf{pass}_{\text{real}} = \mathsf{pass}_{\text{user}} \parallel 0x00 \parallel \mathsf{pass}_{\text{ent}}.$$

The delimiter $0x00$ guarantees injectivity even under variable-length inputs.
The sealing key is derived exclusively from $\mathsf{pass}_{\text{real}}$:

$$K = \mathsf{Argon2id}(\mathsf{pass}_{\text{real}}, \texttt{"MSCIKDF-seal-key"}, t, m, p).$$

This design has three immediate consequences:

- The **user cannot reconstruct the REV alone**, since $\mathsf{pass}_{\text{ent}}$ is required.
- The **enterprise cannot impersonate the user**, since $\mathsf{pass}_{\text{user}}$ is required.
- **Only the combination of both fragments** reconstructs the ADI.

Thus the authorization boundary becomes two-party but with no shared secret between the parties and no stored key material.

7.2 Revocation Operation

Enterprise revocation is executed by deleting $\mathsf{pass}_{\text{ent}}$ from enterprise storage. Because the REV is recoverable only via:

$$\mathsf{REV} = \mathsf{Unseal}(\mathsf{mnemonic},\ \mathsf{pass}_{\text{user}} \| 0x00 \| \mathsf{pass}_{\text{ent}}),$$

removal of $\mathsf{pass}_{\text{ent}}$ makes it computationally infeasible for the user (or any adversary) to reconstruct the sealing key.
Revocation is:

- **stateless** — no persistent key material exists,
- **instantaneous** — no propagation or synchronization is required,
- **unilateral** — the enterprise acts without user cooperation,
- **non-destructive** — the identity (REV) still exists but is inaccessible.

No on-chain transactions, no hardware modules, and no server-side key hierarchies are involved. The mnemonic becomes unusable immediately after revocation.

7.3 Theorem 3: Revocation Security

Theorem 7.1 (Revocation Security)**.** *Let* sealed *be the AES-GCM-SIV ciphertext obtained by sealing* REV *under* $K(\mathsf{pass}_{\text{real}})$*. If the enterprise deletes* $\mathsf{pass}_{\text{ent}}$*, then any party holding only the mnemonic and* $\mathsf{pass}_{\text{user}}$ *cannot recover* REV *except with negligible probability.*

Proof. The AES-GCM-SIV key is:

$$K(\mathsf{pass}_{\text{real}}) = \mathsf{Argon2id}\big(\mathsf{pass}_{\text{user}} \| 0x00 \| \mathsf{pass}_{\text{ent}}, \texttt{"MSCIKDF-seal-key"}, t, m, p\big).$$

After deleting $\mathsf{pass}_{\text{ent}}$, the user is left only with $\mathsf{pass}_{\text{user}}$ and the sealed ciphertext. Recovering REV requires reconstructing K, which would require:

1. guessing $\mathsf{pass}_{\text{ent}}$ (offline brute-force),

2. inverting Argon2id on an unknown password, or
3. breaking AES-GCM-SIV under IND-CCA security.

All three are assumed to be computationally infeasible. Thus the probability of recovering REV without $\mathsf{pass}_{\text{ent}}$ is negligible.
Formally, for any PPT adversary $\mathcal{A}$:

$$\Pr\left[\mathcal{A}(\mathsf{mnemonic}, \mathsf{pass}_{\text{user}}) = \mathsf{REV}\right] \leq \text{negl}(\lambda).$$

□

7.4 Comparison to KMS Revocation

Traditional enterprise KMS systems require:

- long-term storage of master keys or key-wrapping hierarchies,
- server-side state to track revocation,
- HSM/TPM devices to protect persistent secrets,
- propagation of revocation state across devices.

By contrast, MSCIKDF:

- **stores no master keys**,
- **stores no seeds or private keys**,
- **requires no server-side state**,
- **implements revocation purely by deleting a passphrase component**.

Thus MSCIKDF provides enterprise-level lifecycle control without the operational or infrastructural burden typical of centralized KMS deployments. Even in large distributed systems, revocation is trivial, instantaneous, and requires no communication with the Digital Entity.

8 Security Analysis

This section evaluates the security of MSCIKDF under the adversarial model of Section 3.5. We analyze entropy preservation, confidentiality of the Root Entropy Value (REV), mnemonic leakage, domain isolation, revocation security, and identity-preserving post-quantum migration.
Throughout, we assume AES-GCM-SIV is IND-CCA secure, Argon2id is a secure memory-hard password hash, and HKDF-SHA256 behaves as a PRF in the info argument.

8.1 Entropy Preservation

MSCIKDF begins with a uniformly sampled 128-bit root:

$$\mathsf{REV} \leftarrow \{0,1\}^{128}.$$

Each domain-specific seed is derived using HKDF:

$$s_{\mathsf{curve}} = \mathsf{HKDF_Expand}(\mathsf{REV}, \mathsf{DS}_{\mathsf{curve}}, 32).$$

Because HKDF is a randomness extractor and REV is uniform, the output distribution satisfies:

$$s_{\mathsf{curve}} \approx_c U_{256},$$

i.e., computationally indistinguishable from uniform over 256-bit strings.
Furthermore, for any two curves $i \neq j$ with domain-separation labels $\mathsf{DS}_i \neq \mathsf{DS}_j$, the PRF security of HKDF implies:

$$(s_i, s_j) \approx_c (U_{256}, U_{256}),$$

even to an adversary observing all derived public keys. Thus entropy from REV propagates independently to all domains.

8.2 Confidentiality under Fixed Nonce

MSCIKDF uses a fixed nonce $N = 0^{96}$ (or implicit IV). In standard authenticated encryption (like AES-GCM), nonce reuse under the same key allows an adversary to recover the XOR of plaintexts.
However, MSCIKDF utilizes **AES-GCM-SIV**, which satisfies the *Nonce-Misuse Resistance (NMR)* property. For any two encryptions under the same key K and fixed nonce N:

$$C_1 = \mathsf{Enc}_K(N, M_1), \quad C_2 = \mathsf{Enc}_K(N, M_2)$$

If the plaintexts are distinct ($M_1 \neq M_2$), AES-GCM-SIV ensures that C_1 and C_2 remain indistinguishable from random strings. This prevents keystream reuse attacks. Even if an adversary holds two mnemonics generated from the same passphrase (same K) but different identities (different REV), the ciphertexts reveal no information about the plaintexts other than their inequality.
Formally, for any PPT adversary $\mathcal{A}$ with access only to the mnemonic and all public keys:

$$\Pr[\mathcal{A}(\mathsf{mnemonic}) = \mathsf{REV}] \leq \mathrm{negl}(\lambda).$$

8.3 Mnemonic Leakage Resistance

The mnemonic is merely:

$$\mathsf{mnemonic} = \mathsf{BIP39Encode}(N \| C),$$

where BIP39 is a deterministic bijection on byte strings. Thus the mnemonic contains:

- no entropy input,

- no seeds,
- no public/private keys,
- no curve-specific structure.

Since C is computationally indistinguishable from random, the mnemonic leaks no information beyond its length. In particular:

$$\mathsf{mnemonic} \approx_c \mathrm{RandomWords}_{24}.$$

Hence mnemonic leakage alone does not compromise the identity or authorization material.

8.4 Domain and Curve Isolation

Each domain derives its seed from the same REV but with a distinct, structured domain-separation label:

$$s_{\mathsf{domain}} = \mathsf{HKDF_Expand}(\mathsf{REV}, \mathsf{DS}_{\mathsf{domain}}, 32).$$

HKDF behaves as a PRF in the info argument, so for all $\mathsf{DS}_i \neq \mathsf{DS}_j$:

$$\Pr\left[s_i = s_j\right] \approx 2^{-256},$$

and

$$s_i \perp s_j.$$

Thus compromise in any domain cannot leak:

- signature keys for other curves,
- X25519 E2EE keys,
- future PQC key material,
- identity roots,
- authorization material.

Domain isolation ensures that MSCIKDF provides **cryptographic independence** across all identity use cases: signatures, authentication, E2EE, KMS, and multi-chain asset control.

8.5 Revocation Security

Enterprise revocation (Section 6) deletes the enterprise fragment of the authorization secret:

$$\mathsf{pass}_{\text{ent}},$$

preventing reconstruction of the real sealing key:

$$K = \mathsf{Argon2id}(\mathsf{pass}_{\text{user}} \| 0x00 \| \mathsf{pass}_{\text{ent}}, \texttt{"MSCIKDF-seal-key"}, t, m, p).$$

After deletion, the user possesses only:

$$(\mathsf{mnemonic},\ \mathsf{pass}_{\text{user}}),$$

and must break one of:

1. guessing $\mathsf{pass}_{\text{ent}}$ (offline brute force),
2. inverting Argon2id (assumed infeasible),
3. breaking AES-GCM-SIV security.

Thus revocation yields:

$$\Pr[\text{Recover}(\mathsf{REV}) \mid \mathsf{pass}_{\text{ent}} \text{ deleted}] \leq \text{negl}(\lambda).$$

Importantly, revocation requires:

- no key invalidation,
- no device synchronization,
- no revocation lists,
- no persistent secrets.

It is purely a deletion of an enterprise-controlled passphrase fragment.

8.6 PQC Migration

MSCIKDF supports identity-preserving post-quantum augmentation without:

- changing the mnemonic,
- altering the REV,
- rekeying or migrating assets,
- requiring any user-side state.

A PQC seed is derived by assigning a new domain-separation value:

$$s_{\mathsf{PQC}} = \mathsf{HKDF_Expand}(\mathsf{REV}, \mathsf{DS}_{\mathsf{PQC}}, \ell),$$

where ℓ matches the PQC primitive' s required input length.
PRF security and domain isolation imply:

$$s_{\mathsf{PQC}} \perp \{s_{\mathsf{ed25519}}, s_{\mathsf{secp256k1}}, s_{\mathsf{x25519}}, \ldots\}.$$

Thus PQC material:

- is deterministic from the same identity root,
- remains cryptographically independent,
- requires no mnemonic changes,
- preserves backward compatibility,
- preserves forward compatibility under PQC adoption.

MSCIKDF therefore provides a fully backward-compatible identity layer that smoothly transitions to post-quantum cryptography with no user-visible cost.

9 Applications

MSCIKDF provides a general-purpose, deterministic, multi-curve, and seed-storage-free identity substrate suitable for a wide spectrum of cryptographic systems. Because it decouples identity from authorization, and because all domain-specific keys are deterministic functions of a single Atomic Digital Identity (ADI), the primitive applies uniformly across classical, blockchain, enterprise, and post-quantum settings.

9.1 Multi-Chain Wallets and Asset Control

A single REV deterministically yields signing material across heterogeneous ecosystems:

- Ed25519 (Solana and Tendermint-based systems),
- Secp256k1 (Ethereum, Bitcoin, Cosmos),
- X25519 (E2EE and secure channel establishment),
- PQC material for future rollouts.

Thus, MSCIKDF enables:

- unified identity across all chains and curves,
- deterministic wallet reconstruction without storing any seed,
- stateless secret rotation without rekeying assets,
- curve-isolated private keys preventing cross-chain leakage.

The identity layer becomes independent of wallet format or protocol, making MSCIKDF suitable as a cross-chain identity substrate.

9.2 Secure Messaging and End-to-End Encryption

MSCIKDF naturally supports messaging protocols requiring:

- long-term identity keys,
- per-peer or per-session ECDH material,
- deterministic rekeying for multiple devices,
- forward-secret and domain-isolated encryption keys.

The X25519 domain yields deterministic E2EE identity keys, while session keys are derived through standard DH-based ratchets. Because the identity never requires storing a seed, multi-device operation becomes stateless and reproducible from the same mnemonic – passphrase pair.

Note: The xIdentity serves as the static root of trust. Actual communication MUST employ standard ratcheting protocols (e.g., Double Ratchet) utilizing ephemeral keys derived from the ADI to ensure forward secrecy.

9.3 Enterprise KMS and Cloud Identity

Conventional cloud KMS systems require stored master keys, wrapping keys, or hardware security modules (HSMs). MSCIKDF replaces these with:

- a reproducible ADI root (never stored),
- deterministic domain-separated key derivation,
- passphrase splitting for organization – user co-control,
- stateless revocation via deletion of the enterprise component.

Thus an enterprise gains full lifecycle control over identity without maintaining long-term secrets or state, reducing KMS operational complexity while improving auditability.

9.4 AI Agents and Autonomous Digital Entities

Digital Entities—AI agents, autonomous services, digital twins, and large-scale agentic frameworks—require:

- deterministic identity across deployments,
- multi-curve keys for multi-protocol interaction,
- per-peer E2EE communication,
- secure revocation and lifecycle control,
- identity continuity across embeddings, models, or environments.

MSCIKDF provides a unified identity substrate for agents operating across:

- cloud or on-device inference,
- decentralized networks,
- cross-application embeddings,
- long-lived autonomous workflows.

Because the ADI requires no stored seed, autonomous agents can be redeployed, copied, or rescaled deterministically while preserving identity semantics and security guarantees.

9.5 IoT, Robotics, and Cyber-Physical Systems

Physical devices face severe constraints:

- limited secure storage,
- intermittent connectivity,
- multi-curve protocol requirements (TLS, Noise, blockchain),
- device provisioning and revocation challenges.

MSCIKDF enables:

- stateless device provisioning,
- deterministic, non-stored identity roots,
- device – cloud trust anchored in an ADI without hardware storage,
- revocation without firmware updates or key rotation.

Thus MSCIKDF acts as an identity layer for large fleets of robots or IoT devices without requiring embedded secure elements.

9.6 Post-Quantum Migration

PQC-enabling a system typically requires:

- generating new root seeds,
- migrating device or wallet state,
- updating identity across protocols.

In MSCIKDF, PQC material is derived using a new domain-separation label:

$$s_{\mathsf{PQC}} = \mathsf{HKDF_Expand}(\mathsf{REV}, \mathsf{DS}_{\mathsf{PQC}}, \ell),$$

ensuring:

- identity continuity,

- classical – PQC key separation,
- backward-compatible operation,
- no mnemonic or wallet migration.

This provides a practical transition path for both enterprises and decentralized systems anticipating PQC requirements.

10 Practical Migration Helper (Optional)

Although MSCIKDF is defined as a seed-storage-free identity primitive, real-world deployments frequently contain long-lived legacy private keys (e.g., ECDSA keys controlling existing blockchain assets). For convenience, we provide a helper function—not part of the primitive and not affecting the security model—that allows such keys to be wrapped into a Crypto Entity.
Given a legacy private key $\mathsf{sk}_{\text{legacy}}$ and a passphrase pass, the helper function

$$\texttt{wei_to_crypto_entity} : (\mathsf{sk}_{\text{legacy}}, \mathsf{pass}) \longrightarrow \mathsf{mnemonic}$$

produces an MSCIKDF mnemonic whose Crypto Entity is able to reproduce the same legacy address while also obtaining all deterministic multi-curve material defined in MSCIKDF.
This mechanism is purely a practical embedding layer for migration purposes; it does not modify the MSCIKDF construction, introduce new assumptions, or affect any aspect of the formal security analysis. For clarity and simplicity, we omit the implementation details.

11 Implementation

We provide a deterministic reference implementation of MSCIKDF in Rust. The implementation validates the correctness of the construction, enforces memory hygiene, and guarantees cross-platform determinism. All security claims rely solely on the formal specification; the reference library is not part of the security model.

11.1 Reference Library

The core library is written in `no_std`-compatible Rust and implements:

- sampling of a 128-bit Root Entropy Value (REV),
- AES-GCM-SIV sealing and unsealing,
- HKDF-SHA256 expansion with strict domain separation,
- deterministic seed derivation for Ed25519, Secp256k1, and X25519,
- passphrase binding and enterprise revocation.

All sensitive values (REV, curve seeds, AES keys, private keys) exist only ephemerally and are explicitly zeroized upon leaving scope. Stack-local buffers and the `zeroize` trait are used to minimize memory remanence.

11.2 Portability

The library compiles to:

- native platforms (C/FFI consumers),
- WebAssembly,
- iOS and Android mobile targets.

All platforms produce byte-for-byte identical outputs for the same mnemonic – passphrase pair. This determinism holds across architectures and endianness because nonce formats, domain labels, and all encoding conventions are fixed in the specification. No platform stores the REV or any derived secret at rest.

11.3 Test Vectors

A representative test vector is provided to illustrate the deterministic mapping from a mnemonic – passphrase pair to all derived keys. Full multi-curve, Unicode/emoji-supported passphrase, and passphrase-rotation vectors are available in the public repository.

Input.

- Passphrase: `abc`
- Mnemonic: `payment season kiss intact laundry refuse twenty apple sting brown disorder clump sing clap chat various core kiss found monkey place walk leaf bomb`

Solana	`2tJwQGAV4XVzqF7MFnhH7d7U1ugBmaxLJeRu7WGAKv5k`
EVM	`0xe53ed6f2e002e2873370d9cbc664acae1edd3de1`
Bitcoin	`bc1hl5hs5xnfuejmhq3cwhf964w64u8fcc9ffk0xr`
Cosmos	`cosmos1hl5hs5xnfuejmhq3cwhf964w64u8fcc9p2fhhe`
Polkadot	`5ChRNcPNxwnHYJiQc6816RLmEruBWWYmNCYxrDan6GdMhP41`
xidentity	`HACXN6nxwtIdb7dLyYeHfaUmvKqV38WE0IXoiLptLoc=`

Derived Outputs. This vector defines the canonical identity for the given mnemonic – passphrase pair. Any conforming implementation must reproduce these outputs exactly. Additional vectors and full conformance tests are provided at:

`https://github.com/mscikdf/mscikdf-playground`

12 Conclusion

We presented **MSCIKDF**, a cryptographic identity primitive that provides deterministic multi-curve key derivation without storing any long-term secret material. Unlike

hierarchical or seed-based constructions, MSCIKDF removes persistent master seeds entirely, enabling stateless passphrase rotation and unilateral enterprise-controlled revocation while preserving a stable deterministic identity.
MSCIKDF formalizes the notion of an *Atomic Digital Identity (ADI)*: a seed-storage-free root from which independent signing, encryption, authentication, and asset-control domains are derived through structured HKDF labeling. This abstraction captures the identity requirements of emerging *Digital Entities*—AI agents, autonomous services, IoT devices, robots, and multi-chain cryptographic systems—which require portable, reconstructible, and domain-isolated cryptographic identities.
The construction relies solely on well-understood primitives (HKDF, AES-GCM-SIV, Ed25519, Secp256k1, X25519) and inherits their security properties. Because all derived seeds remain isolated through domain separation and the Root Entropy Value is never stored or exported, MSCIKDF significantly reduces key-exposure risks and enables consistent identity reconstruction across platforms and trust boundaries.
By providing a deterministic identity layer that spans heterogeneous cryptographic ecosystems, MSCIKDF unifies functionality that previously required separate mechanisms in wallets, secure messaging protocols, enterprise KMS infrastructure, and autonomous systems. The primitive offers a scalable and future-compatible foundation for cryptographic identity, including smooth post-quantum augmentation without rekeying or state migration.

Future Work. Future directions include formal proofs in the universal composability (UC) framework, integration of standardized post-quantum signature and KEM domains, and exploration of protocol-level applications for machine-to-machine identity and autonomous multi-agent coordination.
Additional directions include defining standardized domain-separation labels and serialization formats for interoperability, and exploring optional hardware-assisted execution (e.g., SGX, TrustZone, HSMs) to further reduce the exposure window of ephemeral secrets.

Normative References

[1] D. McGrew, J. Viega. *AES-GCM-SIV: Nonce-Misuse-Resistant Authenticated Encryption*. RFC 8452, 2019.

[2] H. Krawczyk, P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, 2010.

[3] A. Biryukov, et al. *Argon2: The Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. 2015.

[4] M. Palatinus, et al. *BIP-39: Mnemonic code for generating deterministic keys*. 2013.

[5] A. Langley, et al. *Elliptic Curves for Security (Curve25519)*. RFC 7748, 2016.

[6] S. Josefsson, I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, 2017.

[7] Certicom Research. *SEC 2: Recommended Elliptic Curve Domain Parameters (secp256k1)*. 2010.

Appendix A: Full Pseudocode

Algorithm 5 CombinePassphrase(P_{primary}, $P_{\text{secondary}} \in \{\perp, \text{string}\}$)

```
if P_secondary = ⊥ then
    return P_primary
else
    return P_primary || 0x00 || P_secondary        ▷ NUL separator
end if
```

Algorithm 6 Seal($R \in \{0,1\}^{128}$, passphrase)

```
full ← CombinePassphrase(passphrase, secondary)
salt ← "MSCIKDF-seal-key"                          ▷ Static system-wide salt
K ← Argon2id(password = full, salt = salt, t = 3, m = 64MiB, p = 4)
N ← 0^96                                           ▷ Fixed nonce, secure under AES-GCM-SIV
C ← AES-GCM-SIV_K(N, R)
return C                                           ▷ Returns 32-byte ciphertext only
```

Algorithm 7 DeriveSeeds($R \in \{0,1\}^{128}$)

```
s_ed ← HKDF-SHA256(IKM = R, info = "MSCIKDF-ed25519-seed-v1", 32)
s_secp ← HKDF-SHA256(IKM = R, info = "MSCIKDF-secp256k1-seed-v1", 32)
sk_secp ← Scalar(s_secp) mod n
s_x25519 ← HKDF-SHA256(IKM = R, info = "MSCIKDF-x25519-seed-v1", 32)
Clamp(s_x25519)                                    ▷ Also serves as xIdentity
Zeroize(all temporary buffers)
return (s_ed, sk_secp, s_x25519)
```

Appendix B: Security Proof Sketches

This appendix provides informal game-based reductions supporting the security properties claimed in the MSCIKDF construction. These sketches are not formal proofs but follow standard cryptographic proof techniques.

B.1 Cryptographic Assumptions

The following assumptions are used:

- HKDF-SHA256 behaves as a PRF under HMAC security.
- AES-GCM-SIV provides misuse-resistant IND-CPA and ciphertext integrity (INT-CTXT).
- Argon2id is memory-hard and resists large-scale offline dictionary attacks.
- SHA-256 is collision- and preimage-resistant.
- Ed25519, Secp256k1, and X25519 are secure under standard hardness assumptions.

B.2 Sketch for Theorem 1 (No Seed Persistence)

Game 0 (Real World). The adversary receives $\mathsf{mnemonic} = \mathrm{BIP39Encode}(C)$, where $C = \mathsf{AES\text{-}GCM\text{-}SIV}_K(0, \mathsf{REV})$.

Game 1 (Ideal World). Replace the encryption oracle with a random function. Since AES-GCM-SIV is a Pseudorandom Permutation (PRP) on the input (N, M), and assuming REV is unique, the output C is indistinguishable from a random string $R \leftarrow \{0,1\}^{|C|}$, even with a fixed nonce.

Consequence. The mnemonic is simply an encoding of a value computationally indistinguishable from random noise. Thus, observing the mnemonic provides no advantage in distinguishing the underlying REV or recovering the key.

B.3 Sketch for Theorem 2 (Rotation Soundness)

Secret rotation replaces only the sealing key:

$$K_{\text{old}} = \text{Argon2id}(P_{\text{old}}), \quad K_{\text{new}} = \text{Argon2id}(P_{\text{new}}),$$

while the encrypted payload (the REV) is unchanged.

Argument.

- AES-GCM-SIV authenticity ensures $\mathrm{Dec}_{K_{\text{old}}}(C) = \mathrm{Dec}_{K_{\text{new}}}(C) = \mathsf{REV}$.
- All derived seeds are deterministic HKDF expansions of REV.

Thus rotation preserves all identity-linked material.

B.4 Sketch for Theorem 3 (Revocation Security)

Enterprise binding is:

$$P_{\text{real}} = P_{\text{primary}} \| 0x00 \| P_{\text{secondary}}.$$

Deleting $P_{\text{secondary}}$ prevents reconstruction of the Argon2id key K.

Reduction. An adversary recovering REV without $P_{\text{secondary}}$ would break one of:

- Argon2id memory-hardness,
- HKDF PRF security,
- AES-GCM-SIV authenticated decryption.

Thus enterprise revocation is irreversible under standard assumptions.

B.5 Curve and Domain Isolation

Seeds are derived via:

$$s_{\text{curve}} = \mathsf{HKDF_Expand}(R,\ \mathsf{DS}_{\text{curve}},\ 32).$$

HKDF PRF security in the label ensures:

$$\mathsf{DS}_i \neq \mathsf{DS}_j \quad \Rightarrow \quad s_i \perp s_j.$$

Compromise of one domain does not affect others.

B.6 E2EE Session Confidentiality

Independently derived X25519 seeds ensure:

- forward secrecy through DH ephemeral keys,
- independence from signing or asset keys,
- compartmentalization across protocols.

B.7 PQC Migration Soundness

PQC seeds use a distinct label:

$$s_{\text{PQC}} = \mathsf{HKDF_Expand}(R,\ \texttt{"MSCIKDF-PQC"},\ \ell).$$

Isolation of labels implies:

$$s_{\text{PQC}} \perp s_{\text{classical}}.$$

Because R is unchanged, migration:

- preserves identity continuity,
- requires no mnemonic updates,
- avoids mixed or hybrid secrets,
- is fully backward-compatible.